

ENGAGEMENT AS A DECISION PROBLEM

“A multimodal approach to predict social media post performance”

An Industrial Interaction Report

SUBMITTED TO:-

Dr. Anuj Vora

SUBMITTED BY:-

Lavanya Maithani

B.Tech (CSE)

Graphic Era Hill University

Declaration

I, Lavanya Maithani, a student of B.Tech (Computer Science and Engineering), hereby declare that the industrial interaction report titled “**Engagement as a Decision Problem: A Multimodal Approach to Predict Social Media Post Performance**” is an original record of the work carried out by me under the guidance of Dr. Anuj Vora at Jio Institute, Ulwe, Navi Mumbai.

I further declare that this report, or any part of it, has not been submitted previously to any other university, institute, or organization for the award of any degree, diploma, or certificate. All sources of information, data, and existing research used in the preparation of this report have been duly acknowledged and cited in the References section.

Lavanya Maithani
B.Tech (CSE)
Graphic Era Hill University

Place: Navi Mumbai

Date: August 16, 2026

Acknowledgment

I would like to express my sincere gratitude to my mentor, **Dr. Anuj Vora**, for his invaluable guidance, continuous support, and constructive feedback throughout the course of this industrial interaction project. His insights into machine learning methodology and academic report writing significantly shaped the quality and rigour of this work, and his detailed review comments at every stage were instrumental in improving both the technical depth and presentation of this report.

I am also thankful to **Jio Institute, Ulwe, Navi Mumbai**, for providing the opportunity, infrastructure, and resources necessary to undertake this project, and for facilitating a valuable industrial interaction experience that bridged classroom learning with real-world data science practice.

I extend my appreciation to the faculty and staff of the Department of Computer Science and Engineering at Graphic Era Hill University for their constant encouragement throughout my academic journey.

Finally, I am grateful to my family and friends for their unwavering support and motivation, which enabled me to complete this project successfully.

Lavanya Maithani

Abstract

Social media platforms, particularly Instagram, generate vast amounts of user engagement data every day, yet content creators and businesses typically have no reliable way to estimate how a post will perform before it is published. This report presents a multimodal machine learning approach for predicting Instagram post engagement – specifically, the number of likes a post is expected to receive – by combining account-level attributes, engineered ratio features, and caption-derived textual features into a single predictive framework.

The project follows a complete data science pipeline: raw Instagram data (605,867 posts) is cleaned and preprocessed, five new features are engineered (including follower-following ratio, caption length, word count, hashtag count, and mention count), and five supervised regression algorithms – Linear Regression, Ridge Regression, Lasso Regression, Elastic Net, and Random Forest Regression – are trained on an 80:20 train-test split and compared using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 Score, and 3-fold cross-validation.

Random Forest Regression was identified as the best-performing model, achieving an R^2 score of 0.772 on the held-out test set, substantially outperforming all four linear models, which achieved R^2 scores no higher than 0.135. This result demonstrates that Instagram engagement is driven by non-linear interactions between features that linear models are unable to capture directly. Feature importance analysis further revealed that `comments` and `followers` together account for approximately 73% of the trained model's predictive weight, with engineered features such as `follower_following_ratio` and image-quality attributes contributing meaningfully to the remainder.

The final trained model was deployed through a Flask-based web application that allows a user to input post and account details and instantly receive a predicted like count, an engagement rate, and a qualitative performance rank. This work demonstrates the practical value of combining structured account data, engineered features, and ensemble machine learning methods for real-world social media analytics, and identifies concrete directions for future improvement, including hyperparameter tuning, gradient-boosting models, and richer NLP-based caption analysis.

Keywords: Instagram Engagement Prediction, Machine Learning, Random Forest, Regression Analysis, Feature Engineering, Social Media Analytics, Flask Web Application.

Contents

Declaration	1
Acknowledgment	2
Abstract	3
1 Introduction	6
1.1 Background	6
1.2 Problem Statement	6
1.3 Objectives	7
1.4 Scope of the Project	7
1.5 Motivation	8
1.6 Industrial Relevance	8
2 Literature Survey	10
2.1 Social Media Analytics	10
2.2 Engagement Prediction	10
2.3 Related Work	11
2.4 Research Gap	12
2.5 Comparison of Existing Methods	13
3 System Design and Methodology	14
3.1 System Architecture	14
3.2 Dataset Description	14
3.3 Data Collection	15
3.4 Data Preprocessing	15
3.5 Feature Engineering	16
3.6 Machine Learning Pipeline	17
3.7 Algorithms Used	18
3.7.1 Linear Regression	18
3.7.2 Ridge Regression (L2 Regularization)	18
3.7.3 Lasso Regression (L1 Regularization)	19
3.7.4 Elastic Net	19

3.7.5	Random Forest Regression	19
3.8	Model Training	20
3.9	Cross Validation	21
3.10	Performance Evaluation Metrics	21
3.10.1	Mean Absolute Error (MAE)	21
3.10.2	Root Mean Squared Error (RMSE)	21
3.10.3	R^2 Score (Coefficient of Determination)	22
4	Implementation	23
4.1	Project Folder Structure	23
4.2	Libraries Used	24
4.3	Feature Engineering Script	25
4.4	Model Training Script	26
4.5	Saving Models using Joblib	27
4.6	Feature Importance	28
4.7	Report Generation	29
5	Results and Discussion	31
5.1	Introduction	31
5.2	Model Performance Comparison	31
5.3	Best Model Selection	32
5.4	Feature Importance Analysis	32
5.5	Web Application Output	33
5.6	Limitations Observed	34
5.7	Summary of Findings	35

Chapter 1

Introduction

1.1 Background

Social media platforms have become one of the most influential channels for communication, marketing, entertainment, and business engagement. Among these platforms, Instagram has emerged as a leading visual-content platform where millions of users interact daily through images, videos, reels, and stories. The success of a post on Instagram is commonly measured through engagement indicators such as likes, comments, shares, and user interactions. High engagement not only increases the visibility of content but also plays a crucial role in brand awareness, digital marketing strategies, and audience growth.

With the rapid increase in user-generated content, predicting the performance of a social media post before it is published has become an important research and industrial problem. Several factors influence user engagement, including the quality of the visual content, caption characteristics, hashtags, posting patterns, account popularity, and audience behaviour. The interaction among these factors is highly complex and cannot be effectively understood using traditional statistical methods alone.

Machine learning provides a powerful solution for analysing such multidimensional data and identifying hidden patterns that affect post-performance. By learning from historical engagement data, machine learning models can estimate the expected popularity of future posts and assist content creators in making informed decisions.

This industrial interaction project presents a multimodal machine learning approach for predicting Instagram post-performance by combining account-level attributes, textual information extracted from captions, and image-related features. The project focuses on developing a predictive system capable of estimating the number of likes received by a post using various supervised regression algorithms. The complete workflow includes data preprocessing, feature engineering, model training, evaluation, and comparative analysis of multiple machine learning models.

1.2 Problem Statement

The enormous volume of content published on social media platforms makes it difficult for individuals and organizations to understand which factors contribute to higher engagement. Most users rely on intuition or past experience while creating content, resulting in inconsistent performance and inefficient marketing strategies.

Although Instagram provides engagement metrics after content is published, it does not offer

a mechanism to estimate post-performance beforehand. Existing approaches often focus on a limited set of features such as follower count or posting frequency, while ignoring the combined influence of textual content, account characteristics, and visual information.

The objective of this project is to address this limitation by developing a machine learning-based prediction system capable of estimating Instagram post engagement before publication. By integrating multiple categories of features into a unified prediction framework, the proposed system aims to provide accurate engagement estimates that can support better content creation and strategic decision-making.

1.3 Objectives

The primary objectives of this project are as follows:

- To analyse Instagram post data and identify the factors influencing user engagement.
- To preprocess and prepare the collected dataset for machine learning analysis.
- To perform feature engineering by creating meaningful attributes such as follower-following ratio, caption length, word count, hashtag count, and mention count.
- To develop predictive models capable of estimating the expected number of likes for an Instagram post.
- To train and compare multiple regression algorithms, including Linear Regression, Ridge Regression, Lasso Regression, Elastic Net, and Random Forest Regression.
- To evaluate model performance using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 Score, and Cross-Validation.
- To identify the best-performing machine learning model based on predictive accuracy.
- To generate a reusable prediction model that can assist users in making data-driven content publishing decisions.

1.4 Scope of the Project

The scope of this project covers the complete machine learning pipeline required for predicting Instagram post engagement using structured social media data.

The project includes data loading, preprocessing, handling missing values, encoding categorical variables, feature engineering, model training, evaluation, and model comparison. Multiple regression algorithms are implemented and analysed to determine the most suitable predictive model. The final trained model is stored for future deployment and practical use.

The proposed system can be applied by digital marketers, influencers, businesses, researchers, and content creators to estimate the expected engagement of Instagram posts before publication. Such predictions can support content optimization strategies, improve marketing effectiveness, and reduce uncertainty during campaign planning.

However, the current implementation predicts engagement primarily in terms of the number of likes and relies on historical Instagram data available in the dataset. External factors such as posting time, trending events, seasonal variations, and changes in Instagram's recommendation algorithm are beyond the scope of the present work.

1.5 Motivation

The increasing dependence on social media for marketing, communication, and personal branding has created a growing demand for intelligent systems capable of analysing user engagement. Organizations invest significant resources in creating digital content, yet accurately predicting audience response remains a challenging task.

During the industrial interaction at Jio Institute, the opportunity to work with real-world social media analytics highlighted the practical importance of predictive modelling in supporting data-driven decision-making. This project was motivated by the need to bridge the gap between raw engagement data and actionable insights using modern machine learning techniques.

Another motivating factor was the opportunity to integrate multiple data modalities – including account statistics, textual features, and image-related attributes – into a unified predictive framework. This approach demonstrates how machine learning can transform large volumes of social media data into meaningful predictions that support informed content strategies.

1.6 Industrial Relevance

The ability to predict social media engagement has significant applications across multiple industries. Digital marketing agencies, media organizations, e-commerce companies, influencers, and businesses increasingly rely on analytics to maximize audience engagement and improve campaign performance.

Predictive models enable organizations to evaluate potential content performance before publication, allowing them to optimize captions, hashtags, posting strategies, and overall content quality. Such systems reduce dependence on trial-and-error approaches and facilitate evidence-based marketing decisions.

From an industrial perspective, the techniques implemented in this project – including feature engineering, regression modelling, cross-validation, model evaluation, and automated prediction pipelines – represent core components of modern data science workflows widely adopted across industry. The project demonstrates how machine learning can be effectively applied to real-world social media datasets to generate actionable insights and support strategic

decision-making in digital ecosystems.

Chapter 2

Literature Survey

2.1 Social Media Analytics

Social media analytics is the process of collecting, processing, and analysing data generated on social networking platforms to extract meaningful insights that support decision-making. With the rapid growth of platforms such as Instagram, Facebook, X (formerly Twitter), LinkedIn, and YouTube, organizations increasingly rely on analytics to understand user behaviour, measure campaign effectiveness, and optimize digital marketing strategies.

Instagram has become one of the most influential social media platforms due to its visual-first approach and high user engagement. Every interaction on the platform, including likes, comments, shares, saves, and profile visits, generates valuable data that can be analysed to identify user preferences and content performance.

Recent advances in artificial intelligence and machine learning have significantly enhanced the capabilities of social media analytics. Instead of relying solely on descriptive statistics, modern systems employ predictive models capable of forecasting future engagement based on historical patterns, analysing user profile statistics, textual content, visual features, hashtags, and account characteristics simultaneously [1].

In this project, social media analytics forms the foundation for developing a predictive framework that estimates Instagram post engagement using structured account features, caption-based attributes, and image-related information.

2.2 Engagement Prediction

Engagement prediction refers to the process of estimating how users are likely to interact with social media content before or immediately after it is published. Engagement is commonly measured using indicators such as likes, comments, shares, views, and overall interaction rate.

Several factors influence engagement on Instagram, including the popularity of the account, follower count, posting frequency, caption quality, hashtags, mentions, language, and visual appeal of images. Purba *et al.* [1] showed, using a global dataset of 19,324 posts, that engagement rate decreases as follower count increases – their data showed a decline of roughly 0.026% in engagement rate for every 10,000-follower increase – meaning raw follower count alone is an unreliable, non-linear predictor and must be combined with other feature groups.

Machine learning provides an efficient solution by automatically learning hidden patterns from historical data. Regression algorithms can estimate continuous engagement metrics such as the expected number of likes, while classification algorithms may categorize posts into high-

or low-engagement groups [2].

The present work focuses on predicting the number of likes received by an Instagram post using supervised machine learning regression techniques. To improve prediction accuracy, the project incorporates engineered features such as follower-following ratio, caption length, word count, hashtag count, and mention count, along with account-level and image-related attributes.

2.3 Related Work

Early work on Instagram popularity prediction relied on small, single-source datasets and a limited feature set. De *et al.*, as reported in [1], predicted the popularity of posts for a single lifestyle magazine account using deep learning and achieved up to 88% accuracy – a result that is difficult to generalize, since a single-account dataset has far less variance in engagement rate than a global, multi-account dataset. Zhang *et al.* [3] addressed a global, multi-user dataset directly, framing the task as a binary popular/unpopular classification problem with a dual-attention deep learning architecture and reporting an accuracy of 71.19%, which is lower than single-account studies precisely because of this higher variance.

Purba *et al.* [1] extended this line of work by combining four feature groups – user metadata, post metadata, hashtag features (popularity, visibility, and reachability), and both automated (NIMA-based) and manual image-quality assessment – on a global dataset of 19,324 posts from 16,804 unique users, and compared Linear Regression, Random Forest, and Support Vector Regression to predict Engagement Rate. They found that using only account and post metadata capped the model’s explained variance at $R^2 = 0.426$, that adding automated image features barely moved this figure, but that adding user-history features (engaging followers/outsideers, historical average and standard deviation of engagement rate) raised R^2 to 0.648, and that adding manual image-beauty and artistic-value annotations raised it further to $R^2 = 0.731$ – the best result reported for a global Instagram dataset at the time. Their feature-importance analysis (obtained via Random Forest’s impurity-based ranking, cross-checked against Pearson correlation) further found that hashtag count contributed only marginally to engagement, while image quality and account-history features dominated – a finding directly relevant to the feature set used in the present project.

More recent studies have shifted toward gradient-boosted and deep-learning architectures. Son and Park [4] analysed roughly 6,000 Instagram posts from online travel agencies, combining textual, visual, and social-media-specific features with an XGBoost regressor to identify which feature category most influenced engagement. Kolhe [2] compared Linear Regression, Random Forest, and a Neural Network for average post-engagement prediction, and found that Random Forest achieved the lowest error among the three by handling non-linear feature interactions well, while the Neural Network achieved a marginally better fit at the cost of noticeably higher overfitting. Aswad *et al.* [5] took a sequence-modelling approach, using an LSTM network on a small institutional dataset of 311 posts to combine caption text with contextual features such as

posting time and content type, reporting $R^2 = 0.880$ – though on a dataset small enough that this result should be interpreted cautiously rather than compared directly against studies using tens of thousands of posts.

Across these studies, a consistent pattern emerges: models trained on account metadata alone plateau around $R^2 = 0.4$ – 0.5 , while adding history-based or richer content features (image quality, caption text, hashtag-level statistics) is what pushes performance meaningfully higher. This directly motivates the present project’s approach of combining account-level, engineered ratio, and caption-derived text features within a single regression framework, rather than relying on account metadata in isolation.

2.4 Research Gap

Although considerable progress has been made in social media engagement prediction, several limitations remain in existing approaches.

Many previous studies focus on only one category of features, such as user profile statistics or textual content, while ignoring the combined influence of multiple information sources [1]. Models trained on a limited feature set often fail to capture the complexity of user engagement behaviour, as demonstrated by the low R^2 ceiling (≈ 0.43) Purba *et al.* observed when using metadata alone.

Another limitation is that several studies emphasize classification of engagement into pre-defined categories rather than predicting the actual engagement value [3]. Such categorization reduces the practical usefulness of prediction systems for marketers and content creators who require quantitative estimates.

Furthermore, several existing approaches employ image-annotation pipelines that rely on manual human judges [1] or sequence-based deep learning architectures trained on very small, institution-specific datasets [5], both of which are difficult to scale or deploy in practical, resource-constrained environments. Limited attention has also been given to feature engineering techniques that improve prediction performance while maintaining model interpretability.

The present project addresses these research gaps by integrating multiple categories of engineered features (account-level, ratio-based, and caption-derived) within a computationally efficient, fully automated machine learning pipeline that requires no manual annotation. Instead of relying on a single algorithm, five regression models are evaluated and compared using standard performance metrics including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), R^2 Score, and Cross-Validation. The best-performing model is automatically selected and saved for future deployment.

2.5 Comparison of Existing Methods

Different machine learning algorithms have been applied to predict social media engagement, each offering distinct advantages and limitations. Linear Regression provides a simple and interpretable baseline model but assumes a linear relationship between input features and engagement, limiting its ability to model complex patterns.

Ridge Regression improves upon Linear Regression by incorporating L2 regularization, which reduces overfitting and improves stability when features are highly correlated [6]. Lasso Regression introduces L1 regularization, enabling automatic feature selection by reducing the coefficients of less important variables to zero [7]. Elastic Net combines both L1 and L2 regularization, balancing feature selection with coefficient stabilization [8].

Ensemble learning techniques such as Random Forest Regression construct multiple decision trees and combine their predictions to improve generalization performance [9]. Random Forest models effectively capture nonlinear relationships, automatically model interactions among features, and demonstrate strong resistance to overfitting, and were also the top-performing feature-importance method used by Purba *et al.* [1] in the closest prior study to the present project.

In this project, five regression algorithms – Linear Regression, Ridge Regression, Lasso Regression, Elastic Net, and Random Forest Regression – are implemented and evaluated using the same feature set. Their performance is compared using identical training and testing datasets to ensure a fair evaluation. The model achieving the highest predictive performance is selected as the final engagement prediction model.

The comparative evaluation not only identifies the most suitable algorithm for the current dataset but also provides valuable insights into the strengths and limitations of different regression techniques when applied to Instagram engagement prediction.

Chapter 3

System Design and Methodology

3.1 System Architecture

The proposed Instagram engagement prediction system follows a modular, pipeline-based architecture in which each stage of the machine learning workflow is implemented as an independent component. This design ensures that the system is easy to maintain, test, and extend, since every module performs a specific and well-defined task and communicates with the next stage through intermediate data files stored on disk.

The overall architecture consists of six major stages:

1. **Data Acquisition** – Raw Instagram post and profile data is collected and stored in a structured .parquet file.
2. **Data Preprocessing** – The raw dataset is cleaned, missing values are handled, irrelevant identifier columns are removed, and categorical variables are encoded into numerical form.
3. **Feature Engineering** – New, more informative features are derived from the existing account and caption attributes to strengthen the predictive power of the dataset.
4. **Model Training and Selection** – Multiple regression algorithms are trained on the engineered dataset, evaluated using standard regression metrics, and compared to identify the best-performing model.
5. **Model Persistence** – The trained models, along with the best-performing model, are serialized using `joblib` so that they can be reused without retraining.
6. **Deployment and Presentation** – A Flask-based web application allows a user to input post and account details through a form, after which the trained model predicts the expected number of likes, computes the engagement rate, and assigns a qualitative rank, all of which are displayed on a web interface along with a feature importance visualization.

This layered architecture separates data handling, model logic, and presentation logic, which follows standard machine learning engineering practice and allows individual components (for example, the preprocessing logic or the choice of regression algorithm) to be modified independently without affecting the rest of the system.

3.2 Dataset Description

The dataset used in this project is an Instagram post-level dataset stored in the Apache Parquet columnar format (`instagram.parquet`). Each row of the dataset represents a single Instagram

post along with attributes describing the account that published it and the post's engagement outcome.

The dataset broadly contains three categories of attributes:

- **Account-level attributes:** `followers`, `following`, `num_posts`, `is_business_account`, `lang`
- **Post-level attributes:** `post_type`, `description` (caption text), `description_category`, `description_grade`, `image_grade`
- **Engagement attributes:** `comments` and `likes` (the target variable to be predicted)

In addition, the raw dataset contains several identifier columns such as `sid`, `profile_id`, `shortcode`, `username`, and `path`, which uniquely identify posts and accounts but carry no predictive value. These columns are removed during preprocessing, as discussed in Section 3.4.

The target variable for this project is `likes`, which the system attempts to predict using the remaining account, textual, and image-related features.

3.3 Data Collection

The dataset used for this project was collected in a pre-compiled form and made available as a Parquet file (`instagram.parquet`) rather than being scraped directly from Instagram during this project. The data collection stage of the pipeline is implemented in `preprocess.py`, where the dataset is read using PyArrow's `parquet` module:

```
table = pq.read_table(DATA_PATH)
df = table.to_pandas()
```

Once loaded, the dataset is converted into a Pandas DataFrame, which allows the subsequent preprocessing and feature engineering stages to be carried out using standard data manipulation operations.

3.4 Data Preprocessing

Data preprocessing is a critical stage that converts the raw dataset into a clean, model-ready form. This stage is implemented in `preprocess.py` and consists of the following sequential steps:

1. **Removal of Unnecessary Columns** – Identifier columns that do not contribute to prediction (`sid`, `profile_id`, `shortcode`, `username`, `path`) are dropped from the dataset, since they are unique per record and carry no generalizable predictive signal.

2. **Removal of Duplicate Records** – Duplicate rows are identified and removed using Pandas' `drop_duplicates()` method, ensuring that repeated entries do not bias the model during training.
3. **Handling of Missing Values** – Missing values are handled differently depending on the data type of each column:
 - *Numerical columns* (`int64`, `float64`) – imputed with the **median** of the respective column, since the median is robust to outliers commonly present in engagement data.
 - *Boolean columns* – filled with `False`, a safe default assumption for flags such as `is_business_account`.
 - *Categorical (object) columns* – imputed using the **mode** (the most frequently occurring category) of the respective column.
4. **Encoding of Categorical Variables** – Machine learning algorithms require numerical input, so categorical text columns are converted into numeric codes using Scikit-learn's `LabelEncoder`.
5. **Saving the Processed Dataset** – After cleaning and encoding, the processed dataset is saved as `processed_instagram.csv`, which acts as the input for the next stage of the pipeline – feature engineering.

This preprocessing stage ensures that the dataset entering the feature engineering and modelling stages is complete, consistent, and free from redundant or non-informative fields.

3.5 Feature Engineering

Feature engineering is the process of creating new, more informative variables from existing raw attributes to improve a model's predictive performance. This stage is implemented in `feature_engineering.py`. Five new features are derived, primarily from the account statistics and the caption text (`description` column):

Follower–Following Ratio

```
df["follower_following_ratio"] = (  
    df["followers"] / (df["following"] + 1)  
)
```

This ratio captures the relative “influence” of an account – a high ratio typically indicates a popular or well-established account (many followers, comparatively few follows), which is often correlated with higher engagement.

Caption Length the total number of characters in the post caption, computed as the length of the description string.

Word Count the number of words in the caption, obtained by splitting the caption text on whitespace.

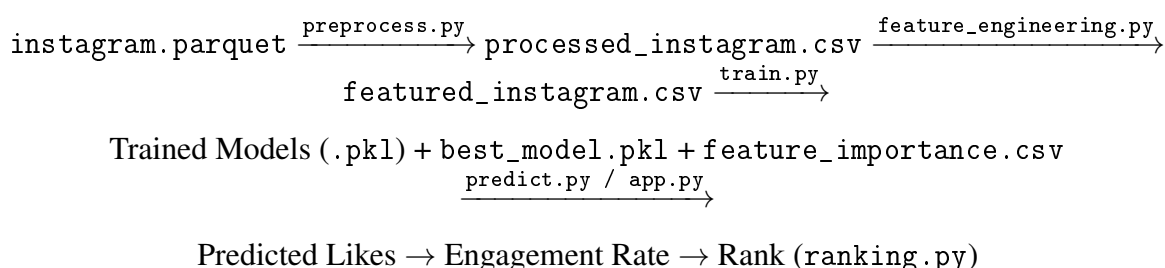
Hashtag Count the number of # symbols present in the caption, used as a proxy for the number of hashtags used.

Mention Count the number of @ symbols present in the caption, representing the number of user mentions or tags.

All text-based features are computed defensively by first filling missing caption values with an empty string (`fillna("")`) to prevent errors during string operations. This feature engineering step reflects the *multimodal* nature of the project – combining numerical account statistics with derived textual signal features into a single unified feature set.

3.6 Machine Learning Pipeline

The complete machine learning pipeline integrates all preceding stages into an end-to-end workflow:



The pipeline uses a log-transformed target variable. Since likes counts are typically right-skewed (a small number of posts receive very high engagement while most receive relatively low engagement), the target is transformed as:

$$y = \log(1 + \text{likes}) \quad (3.1)$$

implemented using numpy's `log1p` function:

```
y = np.log1p(df["likes"])
```

Predictions made by the model are therefore in log-scale and are converted back to the original scale using the inverse transformation $\text{expm1}(y) = e^y - 1$ at prediction time. This transformation stabilizes variance and reduces the influence of extreme outlier posts on model training.

The feature set used consistently throughout training and inference consists of the following fifteen features:

- post_type
- comments
- following
- followers
- num_posts
- is_business_account
- lang
- description_category
- description_grade
- image_grade
- follower_following_ratio
- caption_length
- word_count
- hashtag_count
- mention_count

3.7 Algorithms Used

To identify the most suitable model for predicting Instagram post engagement, five supervised regression algorithms were implemented and compared. Each algorithm represents a different modelling assumption, ranging from simple linear relationships to non-linear ensemble learning.

3.7.1 Linear Regression

Linear Regression is the simplest regression algorithm, modelling the target as a weighted linear combination of the input features:

$$\hat{y} = \beta_0 + \sum_{j=1}^n \beta_j x_j \quad (3.2)$$

where $\hat{y}$ is the predicted log-likes, x_j are the input features (followers, comments, caption length, etc.), β_j are the learned coefficients, and β_0 is the intercept. The coefficients are estimated by minimizing the Residual Sum of Squares (RSS):

$$\text{RSS}(\beta) = \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^n \beta_j x_{ij} \right)^2 \quad (3.3)$$

This loss function has no penalty term, so Linear Regression can overfit when features are correlated (e.g. followers and follower_following_ratio) – which motivates the regularized variants below [10].

3.7.2 Ridge Regression (L2 Regularization)

Ridge Regression extends Linear Regression by adding an L2 penalty on the coefficient magnitudes, shrinking coefficients toward zero without setting them exactly to zero [6]:

$$\text{Loss}_{\text{Ridge}}(\beta) = \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^n \beta_j x_{ij} \right)^2 + \alpha \sum_{j=1}^n \beta_j^2 \quad (3.4)$$

where $\alpha \geq 0$ is the regularization strength. In `train.py`, $\alpha = 1.0$. Larger α shrinks coefficients more, reducing variance at the cost of some bias; Ridge keeps all features in the model, just with smaller weights.

3.7.3 Lasso Regression (L1 Regularization)

Lasso Regression instead applies an L1 penalty, the sum of absolute coefficient values [7]:

$$\text{Loss}_{\text{Lasso}}(\beta) = \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^n \beta_j x_{ij} \right)^2 + \alpha \sum_{j=1}^n |\beta_j| \quad (3.5)$$

In `train.py`, $\alpha = 0.1$. Unlike Ridge, the L1 penalty can shrink coefficients *exactly* to zero, effectively performing automatic feature selection – irrelevant features get dropped from the model entirely.

3.7.4 Elastic Net

Elastic Net combines both L1 and L2 penalties, balancing Ridge’s stability with Lasso’s feature-selection property [8]:

$$\text{Loss}_{\text{EN}}(\beta) = \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^n \beta_j x_{ij} \right)^2 + \alpha \left(\rho \sum_{j=1}^n |\beta_j| + \frac{1-\rho}{2} \sum_{j=1}^n \beta_j^2 \right) \quad (3.6)$$

where ρ (the `l1_ratio`) controls the mix between L1 and L2. In `train.py`, $\alpha = 0.1$ and $\rho = 0.5$, giving an equal blend of both penalties – useful when features are correlated, since Lasso alone tends to arbitrarily pick one of a correlated group and zero out the rest.

3.7.5 Random Forest Regression

Random Forest is an ensemble learning algorithm that builds multiple decision trees on bootstrapped subsets of the training data and averages their predictions [9]. Unlike the linear models above, Random Forest can capture non-linear relationships and complex feature interactions without requiring explicit transformation of the input features.

For a single regression tree, each node t is split by choosing the feature j and threshold s that minimize the weighted variance (impurity) of the resulting child nodes:

$$(j^*, s^*) = \arg \min_{j,s} \left[\frac{N_{t_L}}{N_t} \text{Var}(t_L) + \frac{N_{t_R}}{N_t} \text{Var}(t_R) \right] \quad (3.7)$$

where the variance at a node is:

$$\text{Var}(t) = \frac{1}{N_t} \sum_{i \in t} (y_i - \bar{y}_t)^2 \quad (3.8)$$

This splitting is applied recursively until a stopping condition is reached (e.g. minimum samples per leaf). The prediction at a leaf is the mean target value of the training samples that fall into it. The Random Forest's final prediction averages the predictions of all M individual trees:

$$\hat{y}_{RF} = \frac{1}{M} \sum_{m=1}^M \hat{y}_m \quad (3.9)$$

In `train.py`, the model is configured with $M = 100$ trees (`n_estimators=100`) and `random_state=42` for reproducibility, using all available CPU cores (`n_jobs=-1`) for parallel training. Averaging many trees trained on different bootstrap samples reduces the variance of a single decision tree, which tends to overfit on its own. Random Forest also naturally provides a feature-importance score for each input variable (detailed in Section 4.6), used later to interpret which factors most strongly influence predicted engagement.

The inclusion of both linear (regularized) models and a non-linear ensemble model allows the project to compare interpretability against predictive accuracy, and to determine whether the relationship between account/caption features and engagement is predominantly linear or non-linear in nature.

3.8 Model Training

Model training is implemented in `train.py`. The dataset is split into training and testing subsets using an 80:20 ratio via `train_test_split`, with a fixed `random_state=42` to ensure reproducible results:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

All five algorithms described in Section 3.7 are trained iteratively in a loop, each on the same training split, ensuring a fair comparison. For each model:

1. The model is trained (`model.fit(X_train, y_train)`) on the log-transformed target.
2. Predictions are generated on the held-out test set.
3. Predictions and true values are converted back to the original likes scale using `np.expml()` for interpretability purposes.
4. Evaluation metrics (Section 3.10) are computed, and the model is saved individually to the `models/` directory using `joblib.dump()`.

After all models are trained, the algorithm achieving the highest R^2 score on the test set is selected as the best model and separately saved as `best_model.pkl`, which is subsequently loaded by both `predict.py` and `app.py` for making live predictions. A consolidated comparison of all trained models – including MAE, RMSE, R^2 , and cross-validation score – is saved as `model_results.csv` in the `reports/` directory.

3.9 Cross Validation

To ensure that the reported performance of each model is not simply a result of a favourable train-test split, k -fold cross-validation is applied using Scikit-learn's `cross_val_score` function with $k = 3$, scored using the R^2 metric:

```
cv_score = cross_val_score(
    model, X_train, y_train, cv=3, scoring="r2", n_jobs=-1
).mean()
```

In 3-fold cross-validation, the training data is further divided into three equal parts; the model is trained on two parts and validated on the remaining part, and this process is repeated three times so that each part serves as the validation set exactly once. The average R^2 score across the three folds gives a more robust estimate of how well each model is expected to generalize to unseen data, compared to relying on a single train-test split alone.

3.10 Performance Evaluation Metrics

To objectively compare the performance of the five regression algorithms, three standard regression evaluation metrics are used, all computed on the log-transformed predictions and true values.

3.10.1 Mean Absolute Error (MAE)

MAE measures the average absolute difference between predicted and actual values:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad (3.10)$$

It gives an intuitive measure of the average magnitude of prediction error, without considering the direction (over- or under-prediction) of the error.

3.10.2 Root Mean Squared Error (RMSE)

RMSE measures the square root of the average squared difference between predicted and actual values:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (3.11)$$

Because errors are squared before averaging, RMSE penalizes larger errors more heavily than MAE, making it particularly useful for identifying models that occasionally produce large mispredictions (for example, drastically underestimating a viral post).

3.10.3 R^2 Score (Coefficient of Determination)

The R^2 score measures the proportion of variance in the target variable that is explained by the model's predictions:

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (3.12)$$

An R^2 score close to 1 indicates that the model explains most of the variability in engagement, while a score close to 0 (or negative) indicates poor predictive performance. R^2 is used in this project as the primary criterion for selecting the best-performing model, since it is scale-independent and easily interpretable as a percentage of explained variance.

Together, MAE, RMSE, R^2 , and the cross-validation score provide a comprehensive evaluation framework – MAE and RMSE quantify the magnitude of prediction error, R^2 quantifies overall model fit, and cross-validation verifies that the model's performance generalizes beyond a single data split.

Chapter 4

Implementation

4.1 Project Folder Structure

The project is organized using a modular directory structure that separates raw data, source code, trained models, reports, and the web application front-end. This separation of concerns makes the codebase easier to navigate, maintain, and extend. The overall structure of the project is as follows:

```
Engagement-Predictor/
|-- dataset/
| |-- instagram.parquet # Raw dataset
| |-- processed_instagram.csv # From preprocess.py
| |-- featured_instagram.csv # From feature_engineering.py
|
|-- src/
| |-- preprocess.py # Data cleaning & encoding
| |-- feature_engineering.py # Derived feature creation
| |-- eda.py # Exploratory data analysis
| |-- train.py # Model training & selection
| |-- evaluate.py # Standalone evaluation
| |-- predict.py # Sample prediction script
| |-- ranking.py # Engagement rate & ranking
| |-- app.py # Flask web application
|
|-- models/
| |-- linear_regression.pkl
| |-- ridge.pkl
| |-- lasso.pkl
| |-- elasticnet.pkl
| |-- random_forest.pkl
| |-- best_model.pkl # Auto-selected best model
|
|-- reports/
| |-- correlation_heatmap.png
| |-- likes_distribution.png
| |-- followers_vs_likes.png
| |-- model_results.csv
| |-- feature_importance.csv
|
|-- frontend/
| |-- templates/
| | |-- index.html
| |-- static/
| |-- (CSS / JS / assets)
|
|-- requirements.txt
```

Every script under `src/` derives its project root dynamically using:

```
BASE_DIR = os.path.dirname(
    os.path.dirname(os.path.abspath(__file__)))
```


)

This ensures that file paths for the dataset, models, and reports resolve correctly regardless of the directory from which a script is executed, making the pipeline portable across machines. The `dataset/`, `models/`, and `reports/` directories act as the shared interface between successive stages of the pipeline – each script reads its input from, and writes its output to, one of these common folders, which keeps the individual scripts decoupled from one another.

4.2 Libraries Used

The project relies on a well-established Python data science and web development stack, listed in `requirements.txt`. The key libraries and their roles are summarized in Table 4.1.

Table 4.1: Libraries used in the project and their roles

Category	Library	Purpose
Data Handling	pandas	Loading, cleaning, and manipulating tabular data (DataFrames)
Data Handling	pyarrow	Reading the raw dataset stored in Parquet format
Numerical Computing	numpy	Array operations and mathematical transformations (e.g., <code>log1p</code> , <code>expm1</code>)
Machine Learning	scikit-learn	Preprocessing (LabelEncoder), model implementations (Linear/Ridge/Lasso/ElasticNet/Random Forest), train-test splitting, cross-validation, and evaluation metrics
Gradient Boosting	xgboost, lightgbm, catboost	Included in the environment for potential gradient-boosting extensions to the modelling pipeline
Model Persistence	joblib	Serializing and deserializing trained models to/from <code>.pkl</code> files
Visualization	matplotlib, seaborn	Static plots for exploratory data analysis (heatmaps, histograms, scatter plots)
Visualization	plotly	Interactive feature-importance chart rendered in the web application
Web Framework	Flask	Serving the prediction form and displaying results through a browser-based interface
Supporting	Werkzeug, Jinja2, itsdangerous, click, blinker	Core dependencies of the Flask framework (routing, templating, request handling)

While gradient-boosting libraries such as XGBoost, LightGBM, and CatBoost are included in the project’s environment, the current implementation of `train.py` restricts model comparison to five classical regression algorithms (Linear Regression, Ridge, Lasso, ElasticNet, and Random

Forest). These additional libraries represent a natural direction for future enhancement of the modelling pipeline, as discussed in Chapter 5.

4.3 Feature Engineering Script

The feature engineering stage is implemented in `feature_engineering.py` and executes after preprocessing has produced `processed_instagram.csv`. The script performs the following operations in sequence:

```
df = pd.read_csv(INPUT_PATH)

# Follower-Following Ratio
df["follower_following_ratio"] = (
    df["followers"] / (df["following"] + 1)
)

# Caption-derived features (guarded by column check)
if "description" in df.columns:
    desc = df["description"].fillna("").astype(str)
    df["caption_length"] = desc.apply(len)
    df["word_count"] = desc.apply(
        lambda x: len(x.split())
    )
    df["hashtag_count"] = desc.apply(
        lambda x: x.count("#")
    )
    df["mention_count"] = desc.apply(
        lambda x: x.count("@")
    )

df.to_csv(OUTPUT_PATH, index=False)
```

Two implementation details are noteworthy:

1. **Defensive column checking** – each caption-derived feature block is wrapped in an `if "description" in df.columns:` guard. This prevents the script from crashing if the `description` column is absent or has already been dropped upstream, making the script more robust to changes in the preprocessing stage.
2. **Null-safe string operations** – every operation on the `description` column first applies `.fillna("").astype(str)` before performing string-based computations (`len`, `.split()`, `.count()`). This avoids `TypeError` exceptions that would otherwise occur when calling string methods on `NaN` (float) values.

The output of this script, `featured_instagram.csv`, contains the original cleaned columns plus five newly engineered features, and serves as the direct input to the model training stage.

4.4 Model Training Script

The model training script, `train.py`, orchestrates the comparison of five regression algorithms and the selection of the best-performing model. Its key implementation steps are:

a) Feature and Target Selection

```
features = [  
    "post_type",  
    "comments",  
    "following",  
    "followers",  
    "num_posts",  
    "is_business_account",  
    "lang",  
    "description_category",  
    "description_grade",  
    "image_grade",  
    "follower_following_ratio",  
    "caption_length",  
    "word_count",  
    "hashtag_count",  
    "mention_count",  
]  
target = "likes"  
  
X = df[features]  
y = np.log1p(df[target]) # log-transformed target
```

b) Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.20, random_state=42  
)
```

c) Model Dictionary

All five candidate models are stored in a dictionary, which allows them to be trained and evaluated identically inside a single loop:

```
models = {  
    "Linear Regression": LinearRegression(),  
    "Ridge": Ridge(alpha=1.0),  
    "Lasso": Lasso(alpha=0.1),  
    "ElasticNet": ElasticNet(alpha=0.1, l1_ratio=0.5),  
}
```

```
"Random Forest": RandomForestRegressor(  
    n_estimators=100,  
    random_state=42,  
    n_jobs=-1  
)  
}
```

d) Training Loop

For each model in the dictionary, the script fits the model, generates predictions, converts predictions and true values back to the original scale using `np.expml()`, computes MAE, RMSE, R^2 , and 3-fold cross-validation score, appends the results to a results list, and persists the trained model to disk. The model with the highest R^2 score encountered during the loop is tracked and, at the end of training, saved separately as `best_model.pkl`.

e) Reporting

Once training is complete, the collected results are assembled into a `DataFrame`, sorted by R^2 score in descending order, and exported as `model_results.csv`, giving a clear side-by-side comparison of all five algorithms.

4.5 Saving Models using Joblib

Model persistence in this project is handled using the `joblib` library, which is well-suited for serializing Python objects that contain large NumPy arrays, such as trained Scikit-learn estimators.

Saving a trained model:

```
joblib.dump(model, os.path.join(MODEL_DIR, filename))
```

Each of the five trained models is saved individually inside the `models/` directory, using a filename derived from the model's name (e.g., "Random Forest" → `random_forest.pkl`):

```
filename = name.lower().replace(" ", "_") + ".pkl"
```

In addition to saving each individual model, the single best-performing model (selected based on the highest R^2 score) is saved separately as `best_model.pkl`:

```
joblib.dump(  
    best_model,  
    os.path.join(MODEL_DIR, "best_model.pkl")  
)
```

This separation is important from a deployment perspective: rather than requiring the prediction and web-application scripts (`predict.py` and `app.py`) to know which of the five

algorithms performed best, they can simply load `best_model.pkl` directly:

```
model = joblib.load(MODEL_PATH)
```

This design decouples the model selection logic (which happens once, during training) from the model consumption logic (which happens repeatedly, during prediction), meaning the training pipeline can be re-run and the best model can change without requiring any changes to `predict.py` or `app.py`.

4.6 Feature Importance

For tree-based models such as Random Forest, Scikit-learn exposes a `feature_importances_` attribute after training, which quantifies the relative contribution of each input feature to the model's predictions. This is formally known as **Mean Decrease in Impurity (MDI)**: for every split made across every tree, the reduction in node variance (Equation 3.9 in Section 3.7.5) achieved by splitting on a given feature is recorded, weighted by the fraction of samples that reach that node, and summed across all trees. Formally, the importance of feature j is:

$$\text{Importance}(j) = \frac{1}{M} \sum_{m=1}^M \sum_{t: \text{split on } j} \frac{N_t}{N} \Delta I(t, j) \quad (4.1)$$

where $\Delta I(t, j)$ is the impurity decrease achieved at node t by splitting on feature j , N_t is the number of samples at that node, and M is the number of trees. These values are normalized so that all feature importances sum to 1. For linear models, the analogous (but not directly comparable, since it is scale-dependent) information is available through the `coef_` attribute, which gives the learned weight assigned to each feature.

`train.py` checks which of these attributes the best model exposes and builds a feature-importance/coefficient table accordingly:

```
if hasattr(best_model, "feature_importances_"):
    importance = pd.DataFrame({
        "Feature": features,
        "Importance": best_model.feature_importances_
    })
    importance = importance.sort_values(
        by="Importance", ascending=False
    )

elif hasattr(best_model, "coef_"):
    importance = pd.DataFrame({
        "Feature": features,
        "Coefficient": best_model.coef_
    })
    importance = importance.sort_values(
        by="Coefficient", ascending=False
```

```
)
```

This table is exported as `feature_importance.csv` in the `reports/` directory. It serves two purposes in the project:

1. **Interpretability** – it identifies which account, caption, or image attributes most strongly influence predicted engagement, giving content creators actionable insight rather than a black-box number.
2. **Visualization** – the same table is loaded by `app.py` and rendered as an interactive horizontal bar chart using Plotly Express:

```
fig = px.bar(feature_df, x="Importance", y="Feature",  
             orientation="h", color="Importance",  
             title="Feature Importance")
```

This chart is embedded directly into the web application's results page (`feature_chart = fig.to_html(full_html=False)`), allowing end users to visually understand which factors are driving each prediction.

4.7 Report Generation

The project generates two distinct categories of reports, each serving a different analytical purpose.

a) Exploratory Reports (via `eda.py`)

Before modelling begins, `eda.py` produces three static visualizations from the processed dataset, saved as `.png` files in the `reports/` directory:

- `correlation_heatmap.png` – a Seaborn heatmap of the pairwise correlation matrix between all numerical columns, used to identify multicollinearity and to visually inspect which features correlate most strongly with likes.
- `likes_distribution.png` – a histogram of the `likes` column, used to visually confirm the right-skewed distribution of engagement values that motivates the log-transformation applied in `train.py`.
- `followers_vs_likes.png` – a scatter plot of `followers` against `likes`, used to visually assess the relationship between account size and post engagement.

b) Model Performance Reports (via `train.py`)

After training, two structured `.csv` reports are generated:

- `model_results.csv` – a consolidated table comparing all five algorithms across MAE, RMSE, R^2 , and cross-validation score, sorted by R^2 in descending order. This report allows the best model to be justified quantitatively rather than simply asserted.
- `feature_importance.csv` – the ranked feature contribution table described in Section 4.6, which is also rendered interactively within the Flask web application.

Chapter 5

Results and Discussion

5.1 Introduction

This chapter presents the outcomes obtained after executing the complete machine learning pipeline described in Chapters 3 and 4 – preprocessing, feature engineering, model training, and evaluation – on the Instagram engagement dataset (605,867 posts; 484,693 used for training and 121,174 held out for testing). The results of all five regression algorithms are compared using the metrics defined in Section 3.10 (MAE, RMSE, R^2 , and cross-validation score), and the best-performing model is identified. The chapter also discusses the feature importance analysis, interprets the practical meaning of the results, and reflects on the limitations observed during implementation.

5.2 Model Performance Comparison

After training all five algorithms on the 80:20 train-test split and evaluating them on the held-out test set, Table 5.1 summarizes their performance (values on the log-transformed target, as computed in `train.py`).

Table 5.1: Model performance comparison, sorted by R^2 score (descending)

Model	MAE	RMSE	R^2 Score	Cross-Validation (R^2)
Random Forest	0.481	0.639	0.772	0.774
Linear Regression	0.908	1.245	0.135	0.213
Ridge	0.908	1.245	0.135	0.213
ElasticNet	0.924	1.266	0.106	0.190
Lasso	0.940	1.286	0.077	0.164

Random Forest achieves the highest R^2 score by a wide margin (0.772, versus 0.135 or lower for every linear model), confirming that engagement is influenced by non-linear interactions between features that linear models cannot capture directly – for example, the effect of `hashtag_count` on likes plausibly depends on the account’s `followers`, a conditional relationship linear models cannot represent without explicit interaction terms.

Linear Regression and Ridge produce near-identical metrics ($R^2 = 0.135$ for both). This is expected: with $\alpha = 1.0$, Ridge’s L2 penalty is too weak relative to the scale of this feature set to meaningfully shrink coefficients beyond what unregularized Linear Regression already produces. Lasso performs worst of the five models ($R^2 = 0.077$); since `comments` and `followers` alone account for over 70% of Random Forest’s feature importance (Section 5.4), it is likely that

Lasso's L1 penalty ($\alpha = 0.1$) is shrinking some genuinely useful coefficients toward zero, at the cost of predictive power.

For every model, the test-set R^2 and the 3-fold cross-validation R^2 are close (e.g. Random Forest: 0.772 vs. 0.774), indicating that none of the five models are meaningfully overfitting to the particular 80:20 split used, and that the reported performance is a reliable estimate of generalization to unseen posts.

5.3 Best Model Selection

Based on the comparison in Table 5.1, `train.py` automatically selects the model with the highest R^2 score on the test set and saves it as `models/best_model.pkl`. The best-performing algorithm identified was:

Best Model: Random Forest
Best R^2 Score: 0.7723

This model is subsequently used by both `predict.py` (for standalone sample predictions) and `app.py` (for live predictions through the Flask web interface), demonstrating the practical deployment of the model-selection logic described in Section 4.5.

The selection of the best model purely on the basis of test-set R^2 is a reasonable and interpretable criterion for this project, since R^2 is scale-independent and directly communicates the proportion of variance in engagement explained by the model. However, R^2 alone does not capture the magnitude of prediction error in absolute terms – for that, MAE and RMSE (also reported in Table 5.1) should be considered jointly. Random Forest's MAE of 0.481 (on the log-transformed target) corresponds to substantially tighter predictions than any linear model's MAE of 0.908 or higher, reinforcing the R^2 -based choice.

5.4 Feature Importance Analysis

The feature importance table generated in Section 4.6 (`reports/feature_importance.csv`) reveals which input features most strongly influence Random Forest's predictions. Table 5.2 shows the five highest-ranked features.

These five features together account for approximately 87% of the model's total importance ($0.4101 + 0.3179 + 0.0642 + 0.0400 + 0.0396 = 0.8718$), meaning the remaining ten features in the model collectively contribute only about 13%.

Several observations follow from this ranking:

- `comments` is the single most important predictor, ahead of `followers`. This suggests that the model relies heavily on a co-occurring engagement signal (a post that already has many comments is likely to have many likes) rather than purely structural account features – a

Table 5.2: Top 5 features by Random Forest importance

Rank	Feature	Importance
1	comments	0.4101
2	followers	0.3179
3	num_posts	0.0642
4	image_grade	0.0400
5	follower_following_ratio	0.0396

limitation worth noting, since in a true *pre-publication* prediction setting, comments would not yet be known.

- `followers` is the second most important feature, consistent with the intuitive understanding that an account’s existing audience size is a strong structural driver of how many likes a post can realistically receive, and consistent with the follower-driven engagement-rate trend reported by Purba *et al.* [1].
- The engineered feature `follower_following_ratio` (Section 3.5) ranks fifth, supporting the hypothesis that account “influence” (not just raw follower count) contributes additional predictive signal.
- The remaining caption-derived features (hashtag count, word count, caption length, and mention count) rank well below the top five (each under 0.024 in the full table), indicating that while textual caption characteristics do contribute to the model, account-level and co-engagement signals dominate overall – broadly consistent with Purba *et al.*’s [1] finding that hashtag-based features contributed only marginally to their model.
- `lang` has an importance of exactly 0.0000, meaning Random Forest found no split on this feature that reduced prediction error, most likely because the encoded language variable has very low variance or little relationship with the target in this dataset.

5.5 Web Application Output

The trained best model was integrated into a Flask-based web application (`app.py`), which allows a user to enter post and account attributes through an HTML form and receive an instant prediction. Table 5.3 shows a representative sample input (also used in `predict.py`), and Table 5.4 shows the corresponding system output.

The engagement rate is computed by `ranking.py` as:

$$\text{Engagement Rate} = \frac{\text{Predicted Likes} + \text{Comments}}{\text{Followers}} \times 100 \quad (5.1)$$

and the corresponding rank is assigned using threshold-based bands ($\geq 10\%$ = Excellent,

Table 5.3: Sample input parameters

Input Parameter	Sample Value
Followers	15,000
Following	650
Comments	120
Post Type	1
Is Business Account	Yes

Table 5.4: System output for the sample input

Output	Value
Predicted Likes	1,334
Engagement Rate	9.69%
Rank	Very Good

$\geq 7\%$ = Very Good, $\geq 4\%$ = Good, $\geq 2\%$ = Average, $< 2\%$ = Needs Improvement), giving end users an immediately interpretable, qualitative summary alongside the raw numerical prediction.

5.6 Limitations Observed

While the system successfully demonstrates an end-to-end engagement prediction pipeline, the following limitations were observed during implementation and should be acknowledged:

- **Comments as a leakage-prone feature** – as noted in Section 5.4, `comments` is the single most influential feature in the trained model, but comment counts are not typically known before a post is published. A deployment intended for genuine pre-publication forecasting would need to either exclude this feature or replace it with a comment-count *estimate* rather than the true post-publication value.
- **Proxy-based text features** – hashtag and mention counts are approximated by counting # and @ characters rather than using true NLP-based parsing (e.g., regex-based hashtag extraction or a dedicated tokenizer), which may slightly overcount these features in edge cases (for example, an @ used in an email address within a caption).
- **No hyperparameter tuning** – all models are trained using fixed, reasonable default hyperparameters (e.g., Ridge/Lasso α values, Random Forest `n_estimators`) rather than a systematic search (such as `GridSearchCV` or `RandomizedSearchCV`), meaning the reported performance likely represents a lower bound on what each algorithm could achieve. This may particularly affect Lasso, whose $\alpha = 0.1$ produced the weakest result of the five models (Section 5.2).

- **Single train-test split for final metrics** – although 3-fold cross-validation is used to sanity-check generalization, the headline MAE/RMSE/ R^2 values in Table 5.1 come from a single fixed 80:20 split (`random_state=42`), which may not fully capture the variance in performance across different data splits.
- **Unused gradient-boosting libraries** – XGBoost, LightGBM, and CatBoost are installed in the project environment (Section 4.2) but are not currently included in the model comparison, representing untapped potential for further accuracy improvements, particularly given that comparable prior work [4] used XGBoost successfully on a similar task.
- **Likes-only prediction target** – the system predicts only the number of likes and does not model other engagement signals (shares, saves, reach, or comment sentiment), which would give a more holistic view of “engagement” as referenced in the project title.

5.7 Summary of Findings

The results obtained from this project demonstrate that a multimodal feature set – combining account statistics, engineered ratio features, and caption-derived text features – can be used to build an effective predictive model for Instagram post engagement, with Random Forest reaching $R^2 = 0.772$ against a linear-model ceiling of $R^2 = 0.135$. This gap confirms that ensemble-based non-linear models are substantially better suited to this prediction task than purely linear models, owing to the complex, interacting nature of the factors that drive social media engagement – a pattern consistent with prior work on Instagram engagement prediction [1, 2]. The feature importance analysis further shows that co-engagement (`comments`) and account size (`followers`) jointly dominate the model’s predictions, together contributing roughly 73% of total feature importance, while caption-level textual characteristics contribute a smaller but non-trivial share.

These findings, along with the identified limitations, directly motivate the recommendations for future work: hyperparameter tuning, inclusion of gradient-boosting models, addressing the pre-publication feature-leakage concern around `comments`, and richer NLP-based caption features.

Bibliography

- [1] Kristo Radion Purba, David Asirvatham, and Raja Kumar Murugesan. Instagram post popularity trend analysis and prediction using hashtag, image assessment, and user history features. *International Arab Journal of Information Technology*, 18(1):85–94, 2021. doi: 10.34028/iajit/18/1/10.
- [2] Aditya Kolhe. Predicting instagram post user engagement with machine learning models, 2025. ResearchGate preprint.
- [3] Zhi Zhang, Zhenzhen Zhou, Jian Li, Tong Chen, and Jiebo Luo. How to become instagram famous: Post popularity prediction with dual-attention. In *Proceedings of the IEEE International Conference on Big Data*, pages 2383–2392, Seattle, 2018.
- [4] Hyeryeong Son and Yoo Eun Park. Predicting user engagement with textual, visual, and social media features for online travel agencies’ instagram posts: Evidence from machine learning. *Current Issues in Tourism*, 27(22):3608–3622, 2023. doi: 10.1080/13683500.2023.2278087.
- [5] Hazrul Aswad, Dadang Iskandar Mulyana, and Kastum Kastum. Optimizing contextual features for instagram engagement prediction using long short-term memory (lstm). *TIN: Terapan Informatika Nusantara*, 6(3):303–311, 2025. doi: 10.47065/tin.v6i3.8166.
- [6] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- [7] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.
- [8] Hui Zou and Trevor Hastie. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2):301–320, 2005. doi: 10.1111/j.1467-9868.2005.00503.x.
- [9] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- [10] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009.